# Error-Coordinate Residual Learning for On-Manifold MPC

## How residuals are usually estimated

The standard approach (e.g. GP-MPC [1], KNODE-MPC [2]) works as follows. Fly the drone, record the full state at every timestep, and compare what the nominal model predicted to what actually happened:

$$\tilde{\boldsymbol{\varepsilon}}_k = \mathbf{x}_{k+1} - \hat{\mathbf{x}}^{\text{nom}}_{k+1}, \tag{1}$$

where $\hat{\mathbf{x}}^{\text{nom}}$ is the one-step-ahead prediction from the nominal model. A Gaussian process (GP) or a neural network is then trained to predict $\tilde{\boldsymbol{\varepsilon}}_k$ from the current state and input $(\mathbf{x}_k, \mathbf{u}_k)$. At runtime, the learned correction is added to the nominal prediction before the MPC optimises over it.

But the drone's state is not on a flat space.

## The manifold problem

The position $\mathbf{p} \in \mathbb{R}^3$ and velocity $\mathbf{v} \in \mathbb{R}^3$ are ordinary vectors. The orientation is not. A rotation matrix $R$ must satisfy $R^\top R = I$ and $\det R = 1$; it lives on the *special orthogonal group* $\mathrm{SO}(3)$, a curved three-dimensional surface embedded in $\mathbb{R}^{3\times3}$ $(= \mathbb{R}^9)$.

If $R$ is treated as a flat nine-dimensional vector and we learn over it, the network must implicitly discover the orthonormal constraints of the rotation matrix.

Alternatively, Euler angles use only three numbers but has the singularity issue.

### On-manifold MPC

Lu et al. [3] resolve this with a clean idea. Instead of describing the drone's full pose globally, they describe *how far the drone is from the reference trajectory* using local coordinates that always live in ordinary Euclidean space.

They define two operations:

- $\mathbf{x} \oplus \delta$: move a point on the manifold by a small Euclidean perturbation $\delta$.
- $\mathbf{y} \ominus \mathbf{x}$: measure the local Euclidean difference between two nearby manifold points.

For $\mathrm{SO}(3)$ these are the *matrix exponential* and *matrix logarithm*; for $\mathbb{R}^3$ they reduce to ordinary addition and subtraction.

The **error state** at step $k$ is

$$\delta\mathbf{x}_k = \mathbf{x}_k \ominus \mathbf{x}^d_k \;\in\; \mathbb{R}^9, \tag{2}$$

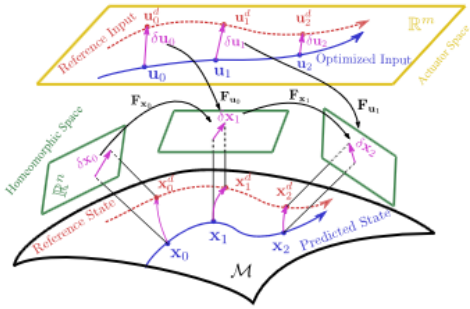



Fig. 4. Interpretation of the error system and its linearization.

Figure 1:

where $\mathbf{x}^d_k$ is the reference. Concretely, for the quadrotor state $\mathbf{x} = (\mathbf{p}, \mathbf{v}, R)$:

$$\delta\mathbf{x}_k = \begin{bmatrix} \mathbf{p}_k - \mathbf{p}^d_k \\ \mathbf{v}_k - \mathbf{v}^d_k \\ \mathrm{Log}\big(R_k^\top R^d_k\big)^\vee \end{bmatrix}, \tag{3}$$

where $\mathrm{Log}(\cdot)^\vee : \mathrm{SO}(3) \to \mathbb{R}^3$ extracts the rotation-error axis-angle vector. The result is a plain nine-dimensional vector with no hidden constraints and no singularities. Linearising the dynamics around the reference gives a standard linear time-varying (LTV) system in $\delta\mathbf{x}_k$:

$$\delta\mathbf{x}_k[+1] \approx F_{\mathbf{x}_k}\delta\mathbf{x}_k + F_{\mathbf{u}_k}\delta\mathbf{u}_k, \tag{4}$$

which is solved by a standard QP solver.

## Residuals in error coordinates

The LTV prediction in (4) still uses a nominal model: the Jacobians $F_{\mathbf{x}_k}$, $F_{\mathbf{u}_k}$ come from rigid-body physics. The true next error state will differ:

$$\boldsymbol{\varepsilon}_k \;=\; \underbrace{\big(\mathbf{x}_{k+1} \ominus \mathbf{x}^d_{k+1}\big)}_{\text{measured}} \;-\; \underbrace{\big(F_{\mathbf{x}_k}\delta\mathbf{x}_k + F_{\mathbf{u}_k}\delta\mathbf{u}_k\big)}_{\text{LTV prediction}} \;\in\; \mathbb{R}^9. \tag{5}$$

This is the **error-coordinate residual**. Comparing it to the standard ambient residual (1):

| | Ambient (standard) | Error coords (ours) |
|---|---|---|
| Rotation representation | $\mathrm{vec}(R) \in \mathbb{R}^9$ | $\delta\boldsymbol{\theta} \in \mathbb{R}^3$ |
| Input dimension | $\mathbb{R}^{19}$ | $\mathbb{R}^{13}$ |
| Input near zero? | No | Yes(because errors) |
| Implicit constraints | 6 | 0 |
| Manifold handling | In loss function | Analytic $\ominus$ pre-step |

The key difference is in the rotation block. Standard methods feed $\mathrm{vec}(R) \in \mathbb{R}^9$ to the network, which must learn across the full curved SO(3) manifold. Our residual replaces this with $\delta\boldsymbol{\theta} \in \mathbb{R}^3$, the *rotation error*, which is always a small vector near zero. The manifold geometry is handled analytically by $\ominus$ before any learning happens.

### Expected Advantage

Because $\delta\mathbf{x}_k$ is always near zero (the controller is trying to track the reference), the input domain of the learning problem is compact and concentrated. A GP prior centred at zero, or a neural network initialised near zero, is well matched to this distribution. An ambient-space model must generalise across all positions, velocities, and orientations the drone might visit. We expect the error-coordinate model to require **fewer training samples** to reach the same residual accuracy.

Model and training architecture are yet to be designed and then the learnt residual is augmented to the nominal model and passed to mpc solver at every step as

$$\delta\mathbf{x}_k[+1] = F_{\mathbf{x}_k}\delta\mathbf{x}_k + F_{\mathbf{u}_k}\delta\mathbf{u}_k + \hat{f}_{\mathrm{res}}\big(\delta\mathbf{x}_k, \delta\mathbf{u}_k, \mathbf{r}_k^d\big)\,. \quad (6)$$

No changes to the QP structure required.

## Hypothesis

For a fixed tracking performance threshold $\tau$ (e.g. position RMSE $< 0.05\,\mathrm{m}$), the error-coordinate model reaches $\tau$ with fewer training samples than an equivalent model trained in ambient coordinates. We test this by sweeping training set size $N$ from 50 to 5 000 samples and plotting closed-loop RMSE for three conditions: nominal (no residual), ambient-space MLP, and ECRL. The advantage should grow with trajectory aggressiveness, because larger attitude excursions make the SO(3) curvature effect more pronounced.

## References


[1] G. Torrente et al., "Data-driven MPC for quadrotors," *IEEE RA-L*, vol. 6, no. 2, pp. 3769–3776, 2021.

[2] K. Y. Chee et al., "KNODE-MPC: A knowledge-based data-driven predictive control framework for aerial robots," *IEEE RA-L*, vol. 7, no. 2, pp. 2819–2826, 2022.

[3] G. Lu, W. Xu, and F. Zhang, "On-manifold model predictive control for trajectory tracking on robotic systems," *IEEE Trans. Ind. Electron.*, vol. 70, no. 9, pp. 9192–9202, 2023.